JAVASCRIPT DOM

Deep & Complete Practical Guide

Basics • Document APIs • Node APIs • Element APIs • Events • Forms • Browser APIs

What makes this edition more complete?

• A structured explanation of the document, node and element interfaces—not just a short list of selectors.

• Detailed coverage of document properties, collections, factories, lookup methods, import/export, writing and event-related APIs.

• Clear differences between similarly named methods, including live vs static collections and node vs element traversal.

• Examples for selection, manipulation, forms, events, storage, observers, accessibility, security and performance.

• A practical reference, projects, revision tables and exercises.

Designed as a study reference. Examples use modern browser JavaScript.

How to study this guide

The **Document Object Model (DOM)** is the browser's object-based representation of an HTML or XML document. JavaScript can use DOM interfaces to inspect the page, change content, create elements, respond to user actions and update the interface.

Language vs API: JavaScript is the programming language. The DOM is a browser API exposed to JavaScript. For example, querySelector() is not a core JavaScript method; it is a method provided by the browser's DOM implementation.

Learning path

Stage	Topics
1. Foundation	HTML tree, document, node types, window/document distinction
2. Finding content	Document selectors, collections, IDs, tags, CSS selectors
3. Navigating	Parent, children, siblings, closest(), matches(), contains()
4. Changing content	Text, markup, attributes, classes, styles
5. Building interfaces	Create, insert, move, clone, replace, remove, fragments
6. Interaction	Events, propagation, delegation, keyboard and pointer events
7. Real applications	Forms, validation, storage, fetch, observers
8. Quality	Accessibility, security, performance, debugging

1. DOM architecture and core concepts

1.1 From HTML source to a DOM tree

The browser parses HTML and creates objects connected in a tree. Elements, text and comments become nodes. The browser then uses document structure and CSS rules to calculate layout and paint the page.

```
<!doctype html>
<html>
  <head><title>Example</title></head>
  <body>
    <main id="app">
      <h1>Hello</h1>
      <p>Learn the DOM.</p>
    </main>
  </body>
</html>

document
 ■■ html
    ■■ head
    ■  ■■ title
    ■     ■■ "Example" (text)
    ■■ body
       ■■ main#app
          ■■ h1
          ■  ■■ "Hello" (text)
```

```
■■ p
   ■■ "Learn the DOM." (text)
```

1.2 Important global objects

Object	Role	Examples
window	Browser window / global browsing context	innerWidth, alert(), setTimeout()
document	Current page document	document.body, querySelector()
navigator	Browser and device capabilities	navigator.language, navigator.onLine
location	Current URL and navigation	location.href, location.pathname
history	Session history	history.back(), history.pushState()
screen	Screen information	screen.width, screen.height

1.3 Node, Element and HTMLElement

A **Node** is the base interface for objects in the document tree. An **Element** is a node representing an element such as div or button. **HTMLElement** is the HTML-specific element interface and adds HTML features such as hidden, title and access to style.

Type	Examples	Key point
Document	document	Represents the whole document
Element	div, svg, custom elements	Represents an element node
HTMLElement	div, input, button	HTML element-specific features
Text	Text between tags	Contains character data
Comment		Comment node
DocumentFragment	Created by createDocumentFragment()	Lightweight container for detached nodes

1.4 Common node types

Node constant	Numeric nodeType	Meaning
Node.ELEMENT_NODE	1	Element
Node.ATTRIBUTE_NODE	2	Attribute (not a normal child in modern DOM trees)
Node.TEXT_NODE	3	Text
Node.COMMENT_NODE	8	Comment
Node.DOCUMENT_NODE	9	Document
Node.DOCUMENT_FRAGMENT_NODE	11	Document fragment

```
const heading = document.querySelector("h1");
```

```
console.log(heading.nodeType); // 1
console.log(heading.nodeName); // H1
console.log(heading.textContent);
```

2. The Document interface — detailed reference

The **Document** interface represents the loaded page. It is the main entry point for finding nodes, creating new nodes, accessing document metadata and working with document-level events. The exact members available can vary slightly by document type and browser support.

2.1 Document identity and metadata properties

Property	What it returns / means	Example
document	Global reference to current document	document === window.document
documentElement	Root element, normally html	document.documentElement
head	The head element	document.head
body	The body element, if present	document.body
title	Document title text	document.title = 'Shop'
URL	Full document URL	document.URL
documentURI	Document URI	document.documentURI
baseURI	Base URL used to resolve relative URLs	document.baseURI
characterSet	Character encoding	document.characterSet
contentType	Document MIME type	document.contentType
doctype	Document type declaration node	document.doctype
readyState	Loading state: loading, interactive or complete	document.readyState
visibilityState	Whether document is visible or hidden	document.visibilityState
compatMode	Standards or quirks mode indicator	document.compatMode
referrer	Referring page URL, if supplied	document.referrer
lastModified	Reported last-modified date string	document.lastModified
defaultView	Window associated with document, if any	document.defaultView

2.2 Document selection methods

Method	Purpose	Return type
getElementById(id)	Find element by its id	Element or null
getElementsByClassName(names)	Find elements by one or more classes	Live HTMLCollection
getElementsByTagName(name)	Find elements by tag name	Live HTMLCollection
getElementsByTagNameNS(ns, name)	Find by namespace and local tag name	Live HTMLCollection
querySelector(selector)	First CSS selector match	Element or null
querySelectorAll(selector)	All CSS selector matches	Static NodeList
getElementsByName(name)	Find elements whose name matches	NodeList (historically/live behavior differs by spec context)

```
const byId = document.getElementById("main");
const byClass = document.getElementsByClassName("card");
const byTag = document.getElementsByTagName("p");
const first = document.querySelector("main .card");
const all = document.querySelectorAll("main .card");
const named = document.getElementsByName("email");
```

2.3 Document creation methods

Method	Purpose	Notes
createElement(tagName, options?)	Create an HTML element	Not inserted until appended
createElementNS(namespace, qualifiedName)	Create element in a namespace	Useful for SVG/XML
createTextNode(text)	Create a text node	Safe for plain text
createComment(text)	Create a comment node	Not displayed as page text
createDocumentFragment()	Create detached fragment container	Useful for batching insertions
createAttribute(name)	Create an Attr object	Usually setAttribute() is simpler
createAttributeNS(ns, name)	Create namespaced attribute	Advanced XML/SVG cases
importNode(node, deep)	Copy node into this document	deep=true copies descendants
adoptNode(node)	Move node into this document	Changes ownerDocument

```
const section = document.createElement("section");
section.id = "news";

const text = document.createTextNode("Latest updates");
section.append(text);

const fragment = document.createDocumentFragment();
fragment.append(section);
document.body.append(fragment);
```

2.4 Document tree and lookup helpers

Method / property	Purpose
elementFromPoint(x, y)	Topmost element at viewport coordinates
elementsFromPoint(x, y)	Stack of elements at viewport coordinates
caretPositionFromPoint(x, y)	Caret position at coordinates where supported
createRange()	Create a Range object for selecting document content
createNodeIterator(root, whatToShow, filter)	Create an iterator through matching nodes
createTreeWalker(root, whatToShow, filter)	Create a navigable filtered tree walker
createEvent(type)	Legacy event factory; prefer new Event() or specific event constructors

```
const topElement = document.elementFromPoint(100, 150);
const range = document.createRange();
const paragraph = document.querySelector("p");

range.selectNodeContents(paragraph);
console.log(range.toString());
```

2.5 Document writing methods — understand before using

Method	Behavior	Guidance
open()	Opens a document stream for writing; can replace current document content	Rarely needed in modern apps
write(markup)	Writes markup into document stream	Avoid for normal DOM updates
writeln(markup)	Like write(), adds a newline	Avoid for normal DOM updates
close()	Closes document stream	Primarily paired with open()/write()

Important: document.write() can implicitly open and replace a document after it has loaded. It can erase the page, interfere with parsing and create security/performance problems. Use createElement(), textContent and append() for normal page updates.

2.6 Document event and state properties

Member	Purpose
activeElement	Currently focused element
fullscreenElement	Element currently in fullscreen mode, if any
pointerLockElement	Element currently holding pointer lock, if any
scrollingElement	Element used for document scrolling in the current mode
hidden	Whether document is hidden
onvisibilitychange	Event handler property for visibility changes (prefer addEventListener)

```
console.log(document.activeElement);

document.addEventListener("visibilitychange", () => {
  console.log(document.visibilityState);
});

document.addEventListener("DOMContentLoaded", () => {
  console.log("HTML parsed and DOM ready");
});
```

2.7 Document methods quick recap

Category	Methods
Find	getElementById, getElementsByClassName, getElementsByTagName, querySelector, querySelectorAll
Create	createElement, createElementNS, createTextNode, createComment, createDocumentFragment
Cross-document	importNode, adoptNode
Inspect position	elementFromPoint, elementsFromPoint
Navigate nodes	createRange, createNodeIterator, createTreeWalker
Legacy writing	open, write, writeln, close

3. Selecting elements and understanding collections

3.1 CSS selector patterns

Selector	Matches
#header	Element with id header
.item	Elements with class item
article	Article elements
section.card	Section elements with class card
nav a	Any a descendant of nav
ul > li	Direct li children of ul
input[type='email']	Email inputs
[data-state='open']	Elements with that data attribute value
h1, h2, h3	Any matching heading in the group
:checked	Checked form controls
:not(.disabled)	Elements that do not match .disabled

3.2 Live HTMLCollection vs static NodeList

A live collection updates when the underlying document changes. A static NodeList is a snapshot of matches at the time querySelectorAll() runs. This distinction matters when adding/removing elements while looping.

```
const live = document.getElementsByClassName("item");
const snapshot = document.querySelectorAll(".item");

console.log(live.length, snapshot.length);

// After adding an element with class item:
// live.length can update automatically.
// snapshot.length remains unchanged.
```

3.3 Iterating safely

```
document.querySelectorAll(".item").forEach((item, index) => {
  console.log(index, item.textContent);
});

// Convert an HTMLCollection to an array when useful:
const items = Array.from(document.getElementsByClassName("item"));
items.forEach((item) => console.log(item));
```

Important: Avoid changing a live collection in a loop when each removal shifts the remaining indexes. Convert it to an array first, or iterate carefully.

4. Node interface — properties and methods

4.1 Node properties

Property	Meaning
nodeType	Numeric type of node
nodeName	Name such as DIV, #text or #document
nodeValue	Character data for some node types; usually null for elements
textContent	Combined text of node and descendants; can also set text
parentNode	Parent node or null
parentElement	Parent if it is an Element, otherwise null
childNodes	Live NodeList of all child nodes
firstChild / lastChild	First or last child node
previousSibling / nextSibling	Adjacent sibling node
ownerDocument	Document that owns this node
isConnected	Whether node is connected to a document
baseURI	Resolved base URL for the node

4.2 Node methods

Method	Purpose
appendChild(node)	Append node as last child; moves it if already elsewhere
insertBefore(newNode, referenceNode)	Insert before a child; null reference appends
removeChild(child)	Remove and return a child node
replaceChild(newChild, oldChild)	Replace a child and return old child
cloneNode(deep)	Clone node; deep=true copies descendants
contains(other)	Check whether node contains another node
hasChildNodes()	Check whether there are child nodes
normalize()	Merge adjacent text nodes and remove empty text nodes
compareDocumentPosition(other)	Describe relative document order/containment
isEqualNode(other)	Compare structure and attributes
isSameNode(other)	Check identity; equivalent to === in modern JS
getRootNode(options)	Return root node; composed option can cross shadow boundaries

```
const list = document.querySelector("ul");
const item = document.createElement("li");
item.textContent = "New";

list.appendChild(item);

const copy = item.cloneNode(true);
list.appendChild(copy);

console.log(list.contains(item));
console.log(item.parentNode === list);
```

4.3 TextContent vs innerText vs innerHTML

Property	Reads	Writes	Caution
textContent	Text of descendants, including hidden text	Replaces children with one text node	Best default for plain text
innerText	Rendered text as a user may see it	Sets rendered text	Can trigger layout calculation
innerHTML	HTML serialization of descendants	Parses supplied string as HTML	Untrusted strings can cause XSS

5. Element and HTMLElement interfaces

5.1 Element properties

Property	Purpose
tagName	Element tag name, typically uppercase for HTML
id	id attribute reflected as a string
className	Class attribute string
classList	DOMTokenList for class manipulation
children	Live collection of child elements only
childElementCount	Number of child elements
firstElementChild / lastElementChild	First/last child element
nextElementSibling / previousElementSibling	Adjacent element sibling
attributes	Live NamedNodeMap of attributes
namespaceURI / localName / prefix	Namespace and element naming information
outerHTML	Serialized element including itself
innerHTML	Serialized child markup
slot	Slot name for slotted content
shadowRoot	Accessible open shadow root, if present

5.2 Element selection and matching methods

Method	Purpose
querySelector(selector)	First matching descendant
querySelectorAll(selector)	Static NodeList of matching descendants
matches(selector)	Whether this element matches CSS selector
closest(selector)	Nearest matching element from self upward
getElementsByClassName(name)	Live descendant collection by class
getElementsByTagName(name)	Live descendant collection by tag
getAttribute(name)	Read attribute string or null
getAttributeNames()	List attribute names
hasAttribute(name)	Check attribute presence
hasAttributes()	Check whether any attributes exist
setAttribute(name, value)	Set attribute
removeAttribute(name)	Remove attribute
toggleAttribute(name, force?)	Add/remove boolean-style attribute
getAttributeNode(name)	Return Attr node
setAttributeNode(attr)	Set Attr node
removeAttributeNode(attr)	Remove Attr node

5.3 Element insertion and replacement methods

Method	Behavior
append(...nodes)	Append nodes or strings at end
prepend(...nodes)	Insert nodes or strings at beginning
before(...nodes)	Insert before element
after(...nodes)	Insert after element
replaceWith(...nodes)	Replace element with supplied nodes/strings
remove()	Remove element from its parent
insertAdjacentElement(position, element)	Insert an element at a relative position
insertAdjacentText(position, text)	Insert text at a relative position
insertAdjacentHTML(position, html)	Parse and insert markup at a relative position
replaceChildren(...nodes)	Remove all children and add supplied nodes

```
const card = document.querySelector(".card");
const badge = document.createElement("span");
badge.textContent = "New";

card.prepend(badge);
card.insertAdjacentText("beforeend", " — updated");
card.replaceChildren(badge);
```

5.4 Geometry and layout methods

Property / method	Meaning
getBoundingClientRect()	Position and size relative to viewport
getClientRects()	List of rectangles occupied by element fragments
scrollIntoView(options)	Scroll ancestor containers to reveal element
scroll()	Scroll element to coordinates/options
scrollTo()	Scroll to absolute coordinates/options
scrollBy()	Scroll by a relative amount
clientWidth / clientHeight	Inner dimensions including padding
offsetWidth / offsetHeight	Layout dimensions including border
scrollWidth / scrollHeight	Full content dimensions including overflow
scrollTop / scrollLeft	Current scroll offsets

5.5 HTML element-specific properties

Property	Purpose
title	Advisory text / title attribute
lang	Language of element content
dir	Text direction
hidden	Whether element is hidden
tabIndex	Keyboard focus order setting
accessKey	Keyboard shortcut hint
draggable	Whether element is draggable
contentEditable	Whether content can be edited
spellcheck	Spellchecking hint
style	Inline CSS declaration object
dataset	Access data-* attributes
inert	Whether subtree is inactive for interaction/accessibility

6. Attributes, properties and data-*

6.1 Attribute vs property

An attribute is written in HTML markup. A property is a JavaScript member on the DOM object. Some attributes reflect to properties. But for dynamic controls, a property can represent the current state while the attribute represents the initial/default state.

```
<input id="name" value="Initial">

const input = document.querySelector("#name");

console.log(input.getAttribute("value")); // initial attribute
console.log(input.value);                 // current live value

input.value = "Changed";
```

6.2 Boolean attributes

For boolean HTML attributes such as disabled, required, checked, selected and hidden, presence generally means true. Setting disabled = false removes the reflected boolean state.

```
const button = document.querySelector("button");
button.disabled = true;
button.disabled = false;

const panel = document.querySelector(".panel");
panel.hidden = true;
```

6.3 dataset

```
<div id="item" data-item-id="25" data-user-role="admin"></div>

const item = document.querySelector("#item");
console.log(item.dataset.itemId);   // "25"
console.log(item.dataset.userRole); // "admin"

item.dataset.status = "active";    // data-status="active"
delete item.dataset.userRole;
```

Important: dataset values are strings. Convert numeric or boolean data explicitly when your application needs those types.

7. Classes, styles and computed CSS

7.1 classList / DOMTokenList

Method / property	Use
add(...tokens)	Add one or more classes
remove(...tokens)	Remove one or more classes
toggle(token, force?)	Toggle or force class presence
contains(token)	Check whether class exists
replace(old, new)	Replace class token
length	Number of tokens
item(index)	Token at index
value	Class string

```
const panel = document.querySelector(".panel");
panel.classList.add("visible", "highlight");
panel.classList.remove("highlight");
panel.classList.toggle("expanded");
panel.classList.toggle("expanded", true);
console.log(panel.classList.contains("visible"));
```

7.2 Inline style methods

```
const box = document.querySelector(".box");

box.style.backgroundColor = "navy";
box.style.padding = "1rem";
box.style.setProperty("--accent", "tomato");
box.style.getPropertyValue("--accent");
box.style.removeProperty("padding");

const computed = getComputedStyle(box);
console.log(computed.display, computed.color);
```

Use CSS classes for reusable states and reserve inline styles for values that genuinely need to be calculated dynamically. getComputedStyle() returns the final computed style and is read-only.

8. Creating and manipulating document content

8.1 Creating nodes

```
const article = document.createElement("article");
const heading = document.createElement("h2");
const text = document.createTextNode("DOM practice");

heading.textContent = "A new article";
article.append(heading, text);
document.querySelector("main").append(article);
```

8.2 Insertion position

Method	Position
parent.append(node)	Inside parent, at the end
parent.prepend(node)	Inside parent, at the beginning
element.before(node)	Immediately before element
element.after(node)	Immediately after element
element.replaceWith(node)	In place of element
insertAdjacentHTML('beforebegin', html)	Before element
insertAdjacentHTML('afterbegin', html)	First child inside element
insertAdjacentHTML('beforeend', html)	Last child inside element
insertAdjacentHTML('afterend', html)	After element

8.3 Moving, cloning and replacing

Appending an existing node moves it; it does not create a second copy. Use cloneNode(true) when a duplicate subtree is required. Be careful with duplicate id values and event listeners: cloneNode() does not copy listeners added with addEventListener().

```
const source = document.querySelector(".card");
const copy = source.cloneNode(true);
copy.removeAttribute("id");
document.querySelector("#cards").append(copy);

const replacement = document.createElement("p");
replacement.textContent = "Replacement content";
source.replaceWith(replacement);
```

8.4 DocumentFragment

A DocumentFragment is a lightweight detached container. When appended, its children move into the destination; the fragment itself does not become a visible wrapper element.

```
const fragment = document.createDocumentFragment();

for (let i = 1; i <= 4; i++) {
  const li = document.createElement("li");
  li.textContent = `Lesson ${i}`;
  fragment.append(li);
}

document.querySelector("ul").append(fragment);
```

9. Events — complete foundation

9.1 addEventListener syntax

```
element.addEventListener(type, callback, options);
```

The browser calls the callback when the selected event occurs. The callback receives an Event object. Prefer addEventListener() over inline HTML event attributes because it separates behavior from markup and supports multiple listeners.

9.2 Common event types

Group	Examples
Pointer/mouse	click, dblclick, mousedown, mouseup, mousemove, mouseenter, mouseleave, pointerdown, pointerup
Keyboard	keydown, keyup
Forms	input, change, submit, reset, invalid, focus, blur
Clipboard	copy, cut, paste
Drag	dragstart, drag, dragenter, dragover, drop, dragend
Window/document	DOMContentLoaded, load, resize, scroll, visibilitychange
Media	play, pause, ended, timeupdate, volumechange
Touch	touchstart, touchmove, touchend

9.3 Event object

Member	Meaning
type	Event name
target	Original event target
currentTarget	Element whose listener is currently executing
eventPhase	Capturing, target or bubbling phase
bubbles	Whether event bubbles
cancelable	Whether default action can be canceled
defaultPrevented	Whether preventDefault() was called successfully
preventDefault()	Cancel default browser behavior when cancelable
stopPropagation()	Stop further propagation
stopImmediatePropagation()	Stop propagation and later listeners on same target
composedPath()	Path event followed through the DOM/shadow DOM

9.4 Listener options

Option	Effect
capture	Listen during capture phase
once	Automatically remove listener after first call
passive	Promise not to call preventDefault(); useful for scroll-related listeners
signal	AbortSignal can remove listener when controller is aborted

```
const controller = new AbortController();

window.addEventListener("resize", () => {
  console.log("Resized");
}, { signal: controller.signal });

// Remove listeners associated with this signal:
controller.abort();
```

9.5 Event delegation

Event delegation uses bubbling: one parent listener handles events from many children. It is useful for dynamic lists because newly added children work without attaching new listeners.

```
const list = document.querySelector("#tasks");

list.addEventListener("click", (event) => {
  const deleteButton = event.target.closest(".delete");
  if (!deleteButton || !list.contains(deleteButton)) return;

  deleteButton.closest("li").remove();
});
```

9.6 DOM ready and script loading

Approach	When to use
script defer	External script runs after HTML parsing, before DOMContentLoaded
DOMContentLoaded	Run after HTML parsed and deferred/module scripts executed
load	Wait for page resources such as images to finish loading
async script	Independent scripts that do not depend on DOM order or other scripts

```
document.addEventListener("DOMContentLoaded", () => {
  console.log("DOM is ready");
});
```

10. Forms, controls and validation

10.1 Useful form control properties

Property	Meaning
value	Current value
defaultValue	Original/default value
checked	Current checkbox/radio state
defaultChecked	Initial checked state
disabled	Whether control is disabled
required	Whether value is required
name	Form submission field name
type	Input/control type
files	Selected files for file input
selectedIndex	Selected option index in select
options	Options collection for select

10.2 Form methods and FormData

Method / API	Purpose
form.submit()	Submits directly without firing submit event or constraint validation
form.requestSubmit()	Submits as if submit button were activated; validation and submit event apply
form.reset()	Resets controls to default values
form.checkValidity()	Checks constraints and returns boolean
form.reportValidity()	Checks and asks browser to report problems
new FormData(form)	Builds key/value data from successful named controls

```
const form = document.querySelector("#signup");

form.addEventListener("submit", (event) => {
  event.preventDefault();

  if (!form.reportValidity()) return;

  const data = new FormData(form);
  console.log(data.get("email"));
});
```

10.3 Constraint validation

Attribute	Purpose
required	Value must be supplied
minlength / maxlength	Text length bounds
min / max	Numeric/date bounds
step	Allowed increments
pattern	Regular-expression constraint for supported text inputs
type=email / url	Built-in format checks

```
const input = document.querySelector("#email");

console.log(input.validity.valid);
console.log(input.validity.valueMissing);
console.log(input.validity.typeMismatch);

input.setCustomValidity("Please check this value.");
input.reportValidity();
input.setCustomValidity(""); // clear custom message
```

Important: Client-side validation improves usability but is not a security boundary. Validate important data on the server as well.

11. Browser storage and URL utilities

11.1 localStorage and sessionStorage

Feature	localStorage	sessionStorage
Lifetime	Persists until cleared	Usually ends when tab session ends
Scope	Same origin	Same origin and tab session
Value type	Strings	Strings
Typical use	Theme preference	Temporary tab workflow state

```
localStorage.setItem("theme", "dark");
const theme = localStorage.getItem("theme");
localStorage.removeItem("theme");

sessionStorage.setItem("step", "2");

// JSON data:
localStorage.setItem("profile", JSON.stringify({ name: "Asha" }));
const profile = JSON.parse(localStorage.getItem("profile") || "null");
```

11.2 URL and URLSearchParams

```
const url = new URL("https://example.com/products?page=2");
console.log(url.hostname);
console.log(url.pathname);

const params = new URLSearchParams(url.search);
console.log(params.get("page"));
params.set("sort", "price");
url.search = params.toString();
```

11.3 fetch and updating the DOM

```
async function loadProducts() {
  const response = await fetch("/api/products");
  if (!response.ok) throw new Error("Could not load products");

  const products = await response.json();
  const list = document.querySelector("#products");

  list.replaceChildren();

  for (const product of products) {
    const li = document.createElement("li");
    li.textContent = product.name;
    list.append(li);
  }
}
```

12. Observers and advanced DOM tools

12.1 MutationObserver

MutationObserver watches configured changes to a DOM subtree. It is useful when code needs to respond to inserted elements or attribute changes.

```
const target = document.querySelector("#app");

const observer = new MutationObserver((records) => {
  for (const record of records) {
    console.log(record.type);
  }
});

observer.observe(target, {
  childList: true,
  subtree: true,
  attributes: true
});

// Later:
observer.disconnect();
```

12.2 IntersectionObserver

IntersectionObserver asynchronously reports when a target intersects a root viewport or container. It is commonly used for lazy loading and revealing content as it enters view.

```
const observer = new IntersectionObserver((entries) => {
  entries.forEach((entry) => {
    if (entry.isIntersecting) {
      entry.target.classList.add("visible");
      observer.unobserve(entry.target);
```

```
    }
  });
});

document.querySelectorAll(".reveal").forEach((el) => observer.observe(el));
```

12.3 ResizeObserver

```
const panel = document.querySelector(".panel");

const resizeObserver = new ResizeObserver((entries) => {
  for (const entry of entries) {
    console.log(entry.contentRect.width, entry.contentRect.height);
  }
});

resizeObserver.observe(panel);
// resizeObserver.disconnect();
```

12.4 Range and Selection

Range represents a portion of document content. Selection represents the text range(s) selected by the user. These APIs are used by editors, annotation tools and text interfaces.

```
const paragraph = document.querySelector("p");
const range = document.createRange();

range.selectNodeContents(paragraph);
console.log(range.toString());

const selection = window.getSelection();
console.log(selection.toString());
```

12.5 NodeIterator and TreeWalker

```
const root = document.querySelector("article");
const walker = document.createTreeWalker(
  root,
  NodeFilter.SHOW_TEXT
);

let node;
while ((node = walker.nextNode())) {
  console.log(node.textContent.trim());
}
```

13. Security, accessibility and performance

13.1 Security

• Use textContent for user-provided plain text.

• Do not concatenate untrusted values into innerHTML or insertAdjacentHTML.

• Validate URLs before placing untrusted values in href or src.

• Do not treat client-side validation as a replacement for server validation.

• Avoid eval() and string-based event handlers.

• Do not store passwords or secret credentials in localStorage or sessionStorage.

13.2 Accessibility

• Use semantic HTML: button, nav, main, header, footer, form and label.

• Give form controls accessible labels.

• Ensure keyboard users can operate interactive controls.

• Keep visible focus indicators.

• Use aria-live sparingly for important dynamic status messages.

• Do not use color as the only way to communicate information.

13.3 Performance

• Cache frequently used selectors when appropriate.

• Use event delegation for large repeated lists.

• Batch creation with DocumentFragment when it improves clarity/performance.

• Avoid layout thrashing: do not alternate layout reads and writes repeatedly in tight loops.

• Use requestAnimationFrame() for animation-related visual updates.

• Debounce expensive search work and throttle high-frequency handlers where suitable.

14. Complete practical projects

Project 1 — Counter

```
<h2 id="count">0</h2>
<button id="minus">Minus</button>
<button id="reset">Reset</button>
<button id="plus">Plus</button>

<script>
let count = 0;
const output = document.querySelector("#count");

function render() {
  output.textContent = count;
}

document.querySelector("#plus").addEventListener("click", () => {
  count++;
  render();
});

document.querySelector("#minus").addEventListener("click", () => {
  count--;
  render();
});

document.querySelector("#reset").addEventListener("click", () => {
  count = 0;
  render();
});
</script>
```

Project 2 — To-do list with event delegation

```
<form id="todo-form">
  <input id="todo-input" name="task" required>
  <button>Add</button>
</form>
<ul id="todo-list"></ul>

<script>
const form = document.querySelector("#todo-form");
const input = document.querySelector("#todo-input");
const list = document.querySelector("#todo-list");

form.addEventListener("submit", (event) => {
  event.preventDefault();
  const value = input.value.trim();
  if (!value) return;

  const li = document.createElement("li");
  const label = document.createElement("span");
  const remove = document.createElement("button");

  label.textContent = value;
  remove.textContent = "Delete";
  remove.type = "button";
  remove.className = "delete";

  li.append(label, " ", remove);
  list.append(li);
  form.reset();
  input.focus();
});

list.addEventListener("click", (event) => {
  const button = event.target.closest(".delete");
  if (button && list.contains(button)) {
    button.closest("li").remove();
  }
});
</script>
```

Project 3 — Live character counter

```
<textarea id="bio" maxlength="150"></textarea>
<p><span id="used">0</span> / 150 characters</p>

<script>
const bio = document.querySelector("#bio");
const used = document.querySelector("#used");

bio.addEventListener("input", () => {
  used.textContent = bio.value.length;
});
</script>
```

Project 4 — Persistent theme toggle

```
<button id="theme-toggle" type="button">Toggle theme</button>

<script>
const toggle = document.querySelector("#theme-toggle");

function applyTheme(theme) {
  document.documentElement.dataset.theme = theme;
  localStorage.setItem("theme", theme);
}

applyTheme(localStorage.getItem("theme") || "light");

toggle.addEventListener("click", () => {
  const current = document.documentElement.dataset.theme;
  applyTheme(current === "dark" ? "light" : "dark");
});
</script>
```

15. Master reference — method index

Interface / area	Important methods and properties
Document	getElementById, getElementsByClassName, getElementsByTagName, querySelector, querySelectorAll, createElement, createElementNS, createTextNode, createComment, createDocumentFragment, importNode, adoptNode, elementFromPoint, elementsFromPoint, createRange, createNodeIterator, createTreeWalker
Node	appendChild, insertBefore, removeChild, replaceChild, cloneNode, contains, hasChildNodes, normalize, compareDocumentPosition, isEqualNode, isSameNode, getRootNode
Element	matches, closest, getAttribute, getAttributeNames, hasAttribute, hasAttributes, setAttribute, removeAttribute, toggleAttribute, append, prepend, before, after, replaceWith, remove, replaceChildren, insertAdjacentElement, insertAdjacentText, insertAdjacentHTML, getBoundingClientRect, getClientRects, scrollIntoView
HTMLElement	focus, blur, click, style, dataset, hidden, tabIndex, contentEditable, title, lang, dir
Events	addEventListener, removeEventListener, dispatchEvent, preventDefault, stopPropagation, stopImmediatePropagation, composedPath
Forms	requestSubmit, submit, reset, checkValidity, reportValidity, setCustomValidity, FormData
Observers	MutationObserver.observe/disconnect, IntersectionObserver.observe/unobserve/disconnect, ResizeObserver.observe/unobserve/disconnect

16. Practice questions and mastery checklist

• Explain the DOM tree and the difference between Node, Element and HTMLElement.

• Describe at least 15 important Document properties and what they return.

• Explain Document selectors and the difference between live and static collections.

• List and explain Document creation methods such as createElement(), createTextNode() and createDocumentFragment().

• What do document.open(), write(), writeln() and close() do, and why should they usually be avoided?

• Compare Node properties such as childNodes, parentNode and textContent.

• Explain appendChild(), insertBefore(), removeChild(), replaceChild() and cloneNode().

• Compare Element children with Node childNodes.

• Explain matches(), closest(), getAttribute() and setAttribute().

• Compare append(), prepend(), before(), after(), replaceWith() and replaceChildren().

• Explain event bubbling, capturing, delegation and listener options.

• Compare form.submit() with form.requestSubmit().

• Explain localStorage, sessionStorage, MutationObserver and IntersectionObserver.

• Why is textContent safer than innerHTML for untrusted text?

• How can keyboard accessibility be maintained when dynamically changing the DOM?

Final learning checklist

Skill	You are ready when you can…
Read the DOM	Inspect document, nodes, elements and collections
Find elements	Use IDs, CSS selectors and scoped queries
Traverse	Move between parent, child and sibling elements
Manipulate	Create, move, clone, insert, replace and remove nodes
Handle events	Build interactions with listeners and delegation
Work with forms	Read values, validate and build FormData
Use browser APIs	Store preferences and observe page changes
Build safely	Protect user content, support keyboard access and avoid costly DOM work

Study advice: Do not memorize only method names. For each method, create a tiny HTML file, run the example, inspect the result in browser DevTools and then change the code. Understanding comes from observing what the browser does.